

Shokunin 職人 AI Ecosystem Guide

Everything you need to know about the open source AI engineering ecosystem for OpenCode. 35 skills, ChromaDB memory, MCP servers, and Windows automation.

Elias Oulkadi · May 2026 · github.com/EliasOulkadi/shokunin

Contents

01 What is Shokunin?

02 Architecture

03 Installation

04 The 35 Skills

05 Memory System

06 MCP Servers

07 Subagents

08 Terminal & Automation

09 Daily Workflows

10 Comparison with Alternatives

11 Quick Reference

What is Shokunin?

Shokunin (職人) is an open source AI engineering ecosystem for Windows. It transforms OpenCode from a blank-slate terminal agent into a production-grade AI assistant with specialized skills, persistent memory, and weekly automations — all for zero cost.

The name comes from the Japanese word for artisan. Each skill aims for that standard: every edge case handled, every workflow automated, every error anticipated.

The ecosystem has four layers that work together:

Interface

OpenCode, VS Code + Cline, WezTerm. You interact through the terminal or your editor.

Agent

35 skills, superpowers plugin, subagents. The AI uses these for domain expertise.

Infrastructure

MCP servers, ChromaDB memory. File access, web fetch, persistent context.

System

Windows 11, PowerShell profile, Task Scheduler. Weekly cleanup and backups.

Everything stays on your machine. No cloud dependencies, no data leaving your PC, no subscriptions.

Architecture

The ecosystem is designed as independent layers. Each can be replaced without affecting the others.

OpenCode · VS Code + Cline · WezTerm
35 skills · superpowers plugin · subagents
MCP: filesystem · fetch · ChromaDB memory
NVIDIA · OpenAI · Anthropic · Ollama (any model)
PowerShell profile · Task Scheduler · Cleanup

OpenCode is the main agent. It reads CLAUDE.md at startup for global instructions, loads skill descriptions, connects to MCP servers, and starts the AI session. When you describe a task, OpenCode matches your words against skill descriptions and activates the right one.

CLAUDE.md contains mandatory instructions including memory search at session start and memory save at session end. This is the key to persistent context.

superpowers plugin adds a complete development methodology: brainstorming, planning, TDD, execution, review, and verification.

Installation

One command installs everything. The installer is idempotent — it creates backups of existing configs and never overwrites without asking.

Open PowerShell and run:

```
irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex
```

What the installer does

Checks prerequisites: PowerShell 5+, Node.js 18+, Python 3.11+, Git

Installs Python dependencies (chromadb)

Copies 35 skills to ~/.config/opencode/skills/

Creates the memory system at ~/.shokunin/

Generates opencode.json with MCP servers, subagents, and superpowers plugin

Installs the PowerShell profile with 20+ aliases

Configures CLAUDE.md with mandatory memory instructions

Creates Windows Task Scheduler entry for weekly maintenance

Asks for NVIDIA API key (free, from build.nvidia.com)

Prerequisites

Software	Install command
Node.js 18+	winget install OpenJS.NodeJS.LTS
Python 3.11+	winget install Python.Python.3.12
Git	winget install Git.Git
OpenCode	npm install -g opencode

Manual setup

Clone the repo and copy the skills:

```
git clone https://github.com/EliasOulkadi/shokunin.git
cp -r shokunin/*/ ~/.config/opencode/skills/
```

The 35 Skills

Each skill is a SKILL.md file that teaches the agent how to handle a specific domain. They auto-activate based on what you ask.

How skills work

At startup, OpenCode reads the name and description from every skill (~100 tokens each). When you send a message, the agent compares your words against all descriptions. If there is a match, the full skill instructions load into context. This is why descriptions are optimized with conversational trigger phrases like "containerize this" or "add login."

Every v3.0 skill includes:

- YAML frontmatter with trigger-optimized description and negative triggers to avoid false activation
- Numbered procedural workflow with step-by-step instructions and decision trees
- Error handling table mapping scenarios to causes and fixes
- Production checklist of verifiable items before marking done
- Anti-patterns table showing what not to do with concrete fixes
- Cited sources (OWASP, NIST, PostgreSQL docs, MDN, etc.)

Advanced skills also include executable PowerShell/Bash scripts, reference documentation, and reusable asset templates.

Skills by domain

Infrastructure

Skill	What it does	Scripts
docker	Multi-stage builds, multi-arch, compose watch, BuildKit cache, vulnerability scanning, seccomp hardening	optimize-dockerfile, scan-image
kubernetes	Gateway API, service mesh (Istio/Linkerd/Cilium), eBPF observability, HPA, PDB, pod debugging	debug-pod, generate-manifest
terraform	Stacks, test framework, moved/removed blocks, pre/postconditions, provider-defined functions, state surgery	validate-all, plan-env
ci-cd	GitHub Actions, GitLab CI, CircleCI, OIDC auth, canary/blue-green deployments, auto-rollback, self-hosted runners	generate-pipeline
db-admin	PostgreSQL backup with verification, health monitoring (7 metrics), PgBouncer pooling, streaming/logical replication, WAL archiving, PITR	backup-db, monitor-db

Backend

Skill	What it does
auth-architect	OAuth2 + PKCE, WebAuthn/Passkeys, JWT with rotation, RBAC/ABAC, OWASP Top 10, rate limiting per endpoint, MFA, session management

api-forge	REST/GraphQL APIs, OpenAPI 3.1 spec, webhooks with HMAC verification, idempotency keys, cursor pagination, rate limiting algorithms
db-sculptor	PostgreSQL index strategy (B-tree, GIN, GiST, BRIN), EXPLAIN ANALYZE, query optimization, expand/contract migrations, Prisma/Drizzle schemas
error-handler	OpenTelemetry tracing, error classification, retry with jitter, circuit breaker, bulkhead, error budgets, burn rate alerts

Frontend

Skill	What it does
component-forge	React/Vue/Svelte components with all states, TypeScript strict, WCAG 2.2, RSC patterns, compound components
responsive-engine	Container Queries, clamp() fluid typography, :has() selector, subgrid, dvh/svh/lvh units, touch targets
motion-craft	Web Animations API (WAAP), Scroll-Driven Animations, FLIP technique, spring physics, motion design tokens
landing-craft	CRO frameworks (LIFT Model, Conversion Research), A/B testing statistics, form optimization, Core Web Vitals
ui-ux-pro-max	Searchable database of UI patterns, color palettes, font pairings, UX guidelines across 13 tech stacks

Mobile

Skill	What it does
flutter	Riverpod, Impeller, Dart 3.7+ patterns, Pigeon platform channels, GoRouter, Clean Architecture
react-native	Expo Router, FlashList, Reanimated 4, New Architecture (Fabric + TurboModules), Hermes, EAS Build

Quality

Skill	What it does
test-commander	Testing Trophy (80% integration), Vitest, Testing Library, MSW, Playwright E2E, visual regression, factory fixtures
performance-profiler	Lighthouse audits, Core Web Vitals optimization, bundle analysis, Node.js profiling, caching strategies

Content & Business

Skill	What it does
communication	Professional emails, SBI/BID feedback model, difficult conversations, meeting notes, cross-cultural communication
content-marketing	Copywriting frameworks (AIDA, PAS, BAB), headline formulas, newsletter deliverability, Cialdini psychology
business-proposals	Cold email sequences, proposals (Good-Better-Best), SOWs with exclusions, 10-slide pitch deck (Sequoia standard)
seo-geo	SEO + Generative Engine Optimization (GEO), llms.txt, schema.org structured data, citability scoring, AI search optimization

translate-craft	8 languages (ES, JA, FR, DE, PT, ZH, KO, AR), cultural adaptation, ICU message syntax, i18next patterns, RTL layout
-----------------	---

Documents, Productivity & System

Skill	What it does
kami	Professional PDF generation with parchment design, 8 templates, Python build scripts, 14 diagram types
git-workflow	Branch/commit/PR/rebase/cleanup automation with 4 PowerShell scripts
windows-powershell	System info, disk cleanup, dev tools install, premium PowerShell profile
memory	ChromaDB persistent memory across sessions
chromadb	Manage the vector database: view, search, delete, backup, reset
strategy, design, documentation, runbook-gen, finance, legal-counsel, portfolio-auto, whendone-plus	Planning, brand, docs, incidents, finance, legal, portfolio, notifications

Memory System

ChromaDB provides persistent vector memory across sessions. The agent saves context at session end and retrieves it at session start automatically.

How it works

The memory system uses ChromaDB, a local vector database. At session start, the agent calls `search_context` to find relevant past entries. At session end, it saves a structured summary with tags, project name, and session identifier. A PowerShell wrapper also saves context automatically after the agent exits, as a fallback.

Memory survives reboots — everything is stored on disk as a sqlite3 file. The base database is ~400 KB including the embedding model. Each entry adds about 1 KB.

Storage

What	Path	Notes
Vector database	~/.shokunin/memory/chroma_db/	400 KB base + ~1 KB per entry
Session logs	~/.shokunin/memory/sessions/	Markdown files for transparency
Weekly backups	~/.shokunin/backups/	Compressed zip, Sunday 21:00
MCP server	~/.shokunin/memory/mcp-server.py	150 lines of Python, JSON-RPC 2.0

Three MCP tools

Tool	Input	Returns
<code>store_context</code>	text, session_id, tags[], project?	Entry ID
<code>search_context</code>	query, project?, tags[]?	Top 5 results with similarity scores
<code>get_session_summary</code>	session_id	All entries for a session

MCP Servers

MCP (Model Context Protocol) servers give OpenCode access to external tools. They start on demand and consume no resources when idle.

Server	Tools	Purpose
filesystem	read_file, write_file, edit_file, search_files, list_directory	Safe file operations with path validation
fetch	fetch, fetch_json	Download web pages and APIs in real time
memory	store_context, search_context, get_session_summary	ChromaDB vector memory

Subagents

Subagents handle specialized tasks in parallel. Each runs with its own model and focus area.

Subagent	Primary model	Fallback (offline)	Purpose
code-review	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Analyze diffs for bugs, security, code smells
debugger	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Root cause analysis from stack traces and logs

Terminal & Automation

PowerShell Profile

Loaded automatically every time you open PowerShell. Includes 20+ aliases, PSReadLine autocomplete, and oh-my-posh prompt with git status.

Category	Aliases
Git	gst, ga, gc, gp, gl, gb, gco
npm	ni, nrd, nrb, nt
Docker	dps, dlog, dstop
Utils	ll, mkcd, touch, which, admin

Automated tasks

Task	Schedule	What it does
Weekly maintenance	Sunday 21:00	Cleans temp files, backs up ChromaDB, checks disk space
Memory save	On OpenCode exit	PowerShell wrapper saves context automatically

Daily Workflows

Describe what you need in natural language. Skills auto-activate through trigger-optimized descriptions.

Building a feature

"Add login to my app" → auth-architect generates OAuth2 + JWT + session management + rate limiting.

"Dockerize it" → docker creates multi-stage Dockerfile + compose + vulnerability scan.

"Deploy to Kubernetes" → kubernetes + ci-cd generate manifests + Gateway API + pipeline with rollback.

Creating content

"Write a blog post" → content-marketing uses PAS framework + headline formulas + SEO structure.

"Export as PDF" → kami generates styled HTML → PDF with parchment design system.

Debugging an incident

"Site is down" → runbook-gen starts incident template + war room + escalation path.

"Check the logs" → error-handler analyzes traces + logs + suggests root cause.

Maintenance

"Check system health" → windows-powershell reports CPU, RAM, disk, uptime.

"Backup the database" → db-admin runs pg_dump + verification + WAL archiving.

Comparison with Alternatives

How Shokunin compares to other AI skill repositories and agent enhancement tools.

Feature	Shokunin	superpowers (obra)	antigravity-awesome	claude-skills
Skills	35 curated	14 methodology	~1300	35
Memory system	ChromaDB ✓	X	X	X
MCP servers	3 included ✓	X	X	X
Installer	One-command ✓	Plugin-based	X	X
Subagents	2 with fallback ✓	✓	X	X
CI/CD	GitHub Actions ✓	✓	X	X
Cost	Zero ✓	Zero	Zero	Zero
Windows automation	Task Scheduler ✓	X	X	X
Executable scripts	27 ✓	Some	X	254 ✓
Stars	0	188K	37K	5.2K

Differentiator: Shokunin is the only project that combines skills + persistent memory + MCP servers + installer + Windows automation in a single package. Competitors do one thing well. Shokunin does everything together.

Quick Reference

Key locations

What	Path
Skills	~/.config/opencode/skills/
OpenCode config	~/.config/opencode/opencode.json
Global AI instructions	~/.claude/CLAUDE.md
Memory data	~/.shokunin/memory/
Memory backups	~/.shokunin/backups/
PowerShell profile	\$PROFILE
Bookmarklet	~/shokunin-bookmarklet.html
Dashboard	~/shokunin-dashboard.html
GitHub repo	github.com/EliasOulkadi/shokunin

Quick commands

Action	Command
Start OpenCode	opencode
Git status	gst
Git add + commit	ga + gc "message"
Git push	gp
npm install	ni
npm run dev	nrd
Open dashboard	start ~/shokunin-dashboard.html

Tech stack

Category	Technology
Agent runtime	OpenCode
Vector DB	ChromaDB (PersistentClient, sqlite)
MCP protocol	JSON-RPC 2.0 over stdin/stdout
Shell	PowerShell 5.1+
Plugin	superpowers (obra/superpowers)
Scheduler	Windows Task Scheduler
Terminal	WezTerm (optional)

Install: irm <https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1> | iex

Repo: github.com/EliasOulkadi/shokunin

License: MIT